UNIDAD 5:

Gestión de Procesos

Máquinas de Estado Finito (FSM) en la Programación

1. Introducción

En el desarrollo de software, a menudo nos encontramos con sistemas que deben comportarse de manera diferente según su situación actual. Una **Máquina de Estado Finito (FSM**) es un modelo computacional que describe un sistema que puede estar en exactamente un estado a la vez de un número limitado (finito) de estados.

El sistema cambia de un estado a otro (transición) en respuesta a entradas o eventos externos.

2. Componentes de una Máquina de Estados

Para definir una FSM, necesitamos identificar cuatro elementos clave:

1. **Estados (S):** Las condiciones o situaciones posibles en las que puede encontrarse el programa (ej: REPOSO, PROCESANDO, ERROR).
2. **Entradas/Eventos (E):** Estímulos que llegan al sistema (ej: presionar un botón, recibir un dato, un temporizador que llega a cero).
3. **Transiciones (T):** Las reglas que indican cómo pasar de un estado a otro según la entrada recibida.
4. **Acciones (A):** Lo que el programa ejecuta al entrar, salir o permanecer en un estado.

3. Representación Gráfica: El Diagrama de Estados

Antes de codificar, se utiliza el **Diagrama de Estados**. En él:

- Los **círculos** representan los estados.
- Las **flechas** representan las transiciones.
- Las **etiquetas sobre las flechas** indican el evento que dispara el cambio.

Ejemplo rápido: Una lámpara inteligente

- Estado A: `APAGADA`.
- Evento: Presionar interruptor -> Transición al Estado B: `ENCENDIDA`.

4. Implementación en Código

La forma más común y eficiente de implementar esto es mediante la estructura de control *switch* (o match en lenguajes modernos) combinada con un enum (enumerado).

Ejemplo en Pseudocódigo / C-like:

C++

```
enum Estado { INACTIVO, CARGANDO, LISTO, ERROR };
Estado estadoActual = INACTIVO;

void actualizarSistema(Evento evento) {
    switch (estadoActual) {
        case INACTIVO:
            if (evento == BOTON_INICIO) {
                iniciarCarga();
                estadoActual = CARGANDO;
            }
            break;

        case CARGANDO:
            if (evento == CARGA_COMPLETA) {
                estadoActual = LISTO;
            } else if (evento == FALLO_SENSOR) {
                estadoActual = ERROR;
            }
            break;

        case LISTO:
            // Lógica para estado listo
            break;

        case ERROR:
            if (evento == BOTON_RESET) {
                estadoActual = INACTIVO;
            }
            break;
    }
}
```

5. Ventajas de usar Máquinas de Estado

- **Claridad:** Evita el "código espagueti" lleno de if-else anidados difíciles de leer.
- **Mantenibilidad:** Es fácil agregar un nuevo estado sin romper la lógica existente.
- **Depuración:** Si el sistema falla, es más sencillo identificar en qué estado se encontraba y qué evento causó la transición incorrecta.
- **Predictibilidad:** El sistema tiene un comportamiento determinista; sabemos exactamente qué hará ante cualquier entrada.

6. Aplicaciones Comunes

- **Videojuegos:** Inteligencia artificial (Patrullar -> Perseguir -> Atacar).
- **Protocolos de Comunicación:** Establecer conexión -> Transmitir -> Cerrar.
- **Interfaces de Usuario (UI):** Formularios con pasos (Paso 1 -> Paso 2 -> Confirmación).
- **Sistemas Embebidos:** Control de semáforos, ascensores o electrodomésticos

Tarea Sugerida:

Diseñar el diagrama de estados para un **Cajero Automático (ATM)** considerando los estados: ESPERANDO_TARJETA, VERIFICANDO_PIN, SELECCIONANDO_MONTO y ENTREGANDO_DINERO.

¿Qué eventos dispararían las transiciones?

Resolución:

1. Definición de Componentes

Estados (S):

1. **ESPERANDO_TARJETA**: Estado de reposo (Idle).
2. **VERIFICANDO_PIN**: El usuario ya ingresó la tarjeta y el sistema valida su identidad.
3. **SELECCIONANDO_MONTO**: Acceso concedido, el usuario elige cuánto dinero extraer.
4. **ENTREGANDO_DINERO**: El hardware procesa la salida de billetes.

Eventos / Entradas (E):

- **TARJETA_INSERTADA:** El sensor detecta el ingreso físico.
- **PIN_CORRECTO:** La validación con la base de datos es exitosa.
- **PIN_INCORRECTO:** Error en la validación.
- **MONTO_VALIDO:** El usuario elige una cifra y tiene saldo suficiente.
- **EXTRACCION_FINALIZADA:** El sensor de salida confirma que el dinero fue tomado.
- **CANCELAR:** El usuario decide abortar la operación.

2. Matriz de Transiciones

Estado Actual	Evento	Estado Siguiente	Acción
ESPERANDO_TARJETA	TARJETA_INSERTADA	VERIFICANDO_PIN	Solicitar PIN en pantalla.
VERIFICANDO_PIN	PIN_CORRECTO	SELECCIONANDO_MONTO	Mostrar menú de cuentas.
VERIFICANDO_PIN	PIN_INCORRECTO	ESPERANDO_TARJETA	Expulsar tarjeta y mostrar error.
VERIFICANDO_PIN	CANCELAR	ESPERANDO_TARJETA	Expulsar tarjeta.
SELECCIONANDO_MONTO	MONTO_VALIDO	ENTREGANDO_DINERO	Contar billetes.
SELECCIONANDO_MONTO	CANCELAR	ESPERANDO_TARJETA	Expulsar tarjeta.
ENTREGANDO_DINERO	EXTRACCION_FINALIZADA	ESPERANDO_TARJETA	Imprimir ticket y expulsar tarjeta.

3. Implementación Sugerida (C++ / Java / C#)

Bibliografía

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la Teoría de Autómatas, Lenguajes y Computación.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software

Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach (9th ed.). New York: McGraw-Hill.

Samek, M. (2008). Practical UML Statecharts in C/C++: Event-Driven Programming for Embedded Systems.

Sommerville, I. (2015). Software Engineering (10th ed.). Boston: Pearson.